AGENTIC VERIFIED OPTIMIZATION OF REAL-WORLD SOFTWARE

Zhe Ye[1]*, Udaya Ghai[2], Zhizhen Qin[2], Dawn Song[1], Soonho Kong[2]
[1]University of California, Berkeley, [2]Amazon Web Services

ABSTRACT

Program optimization aims to improve performance while preserving behavior. Coding agents can already make programs faster by changing algorithms, data structures, and execution strategies. However, existing equivalence checks mostly rely on tests, which agents have been shown to exploit, passing the tests without preserving the intended behavior. We propose *agentic verified optimization*, in which an agent produces a faster implementation together with a machine-checked proof of equivalence to the existing one. Because the existing implementation serves as the specification, the task avoids the specification-writing bottleneck of verified code generation, and recent advances in theorem proving make the proofs attainable. We formulate the task at the function and repository levels in Lean 4, where programs can be both executed and reasoned about. Few real-world applications are written in Lean, so we develop AQUEDUCT, an agentic framework that migrates C, C++, Go, and Rust programs into native Lean, validates each port by differential testing against its original, and then optimizes the ports. On the 189 reference implementations in Verina, agents produce faster proved rewrites for 156, including algorithmic changes such as replacing exponential recursion with dynamic programming. On 17 migrated applications, their 92 proved rewrites speed up 47 of 51 application scenarios. These application-level gains are smaller than the function-level ones, making the choice of functions to optimize an important problem. Current agents can thus already carry out verified optimization on both standalone functions and realistic Lean applications. We discuss the remaining challenges and the steps toward verified optimization in production.

1 INTRODUCTION

Coding agents are increasingly used to write and modify software [6, 30, 82]. They are now also asked to make code faster [26, 18, 74]. This goal continues a long line of work on automatic optimization. Before language models, superoptimizers searched instruction sequences [47, 62], learned models guided search over programs [5], and reinforcement learning discovered faster algorithms [46]. Language models and coding agents have since taken on the task, as PIE [65] rewrites programs for speed, KernelBench [53] studies efficient GPU kernel generation, and Jitskit [40] synthesizes systems specialized to a workload. These optimizations range from local changes that eliminate redundant computation to new data representations and algorithms. Whatever the strategy, a faster replacement must preserve the behavior of the program it replaces.

Agents can already find substantial performance gains, but existing work checks behavior preservation with tests, which, as Dijkstra [16] observed, can show the presence of errors but not their absence. This limitation matters more when the tests also define an agent's reward. Agents then learn to pass the tests rather than preserve the behavior, a pattern studied as reward hacking [67] and observed in coding agents that special-case the tested inputs [92]. In optimization, reward hacking yields speedups from programs that do not preserve behavior. Jitskit [40] describes an apparent $6\times$ improvement that came from regenerating test values rather than storing them, and an audit found correctness failures in 39.5% of the GPU kernels that an optimization harness had accepted [63]. A proof of equivalence avoids this problem because it covers every input, the criterion used in translation validation [55, 50]

*Work done during internship at Amazon Web Services.

and regression verification [24, 10]. With a proof as the check, an agent can change algorithms and data representations freely without weakening correctness.

Agents are now capable provers. Neural theorem proving has progressed from language-model proof search [56, 34] to provers trained with reinforcement learning that reach olympiad level [27, 57, 38], and agents now formalize research mathematics, from open Erdős problems [68, 72] to a complete Lean proof of Fermat's Last Theorem [7, 2]. They also build verified software in Lean, from repository-level implementations with proofs [84] to a verified compiler [58] and a verified DEFLATE library [49]. Program optimization is a natural application of these capabilities.

We study *agentic verified optimization*, in which an agent produces a faster implementation f' of a reference function f together with a machine-checked equivalence proof. The proof establishes $\forall x,\ f(x) = f'(x)$, by using the original implementation f as the specification for correctness. This avoids the specification-writing problem in verified code generation, where writing specifications that capture the intended behavior remains difficult, even manually [19, 86, 45]. This task can also be viewed as the refinement step that formal methods have long advocated, in which a simple implementation is proved correct and then refined into a fast one that inherits its guarantees. By hand, it is costly and has been reserved for critical domains such as cryptography [21]. Agents are a natural fit for this work. In verified code generation, proofs can push agents toward simple implementations [84]. Agentic verified optimization keeps the simple, proved version and then allow agent to search complicated, optimized version with the same correctness guarantee. The task in turn helps improve agents. Every function in an existing codebase is an instance with its specification in place, and because correctness is fixed while speedup has no upper bound, the task is open-ended and does not saturate as agents improve.

The task couples implementation and proof search. For example, replacing recursive enumeration with dynamic programming can remove repeated work, but its proof needs an invariant connecting table entries to the original recursion. An implementation may be equivalent and fast yet difficult for the agent to prove within a limited search budget. Conversely, a readily proved rewrite may offer little improvement. The agent must decide whether to improve a candidate further, invest in its proof, or pursue another implementation. Verified optimization therefore tests how agents coordinate program reasoning, performance analysis, and proof construction.

We formulate the task in Lean 4 [14], where the same definitions can be executed and used in proofs, and whose compiler produces native code with a reference-counting runtime [73]. Lean's kernel checks each equality proof, and execution on the evaluation workloads measures the speedup. We study current agent capabilities at two levels. *Function-level optimization* asks the agent to improve a given implementation. We use the 189 reference implementations in Verina [85] to study this setting. *Repository-level optimization* asks the agent to improve an application by choosing and rewriting functions within its codebase. Here, function-level microbenchmarks measure individual rewrites, while application-level macrobenchmarks measure whether they speed up the application. This setting requires executable applications in Lean, but few real-world applications are written in Lean. We therefore build AQUEDUCT, an agentic framework that migrates C, C++, Go, and Rust programs into idiomatic Lean and checks each port against its original through differential testing [48]. The resulting ports keep the code structure and workloads of real software and serve as targets for verified optimization.

On Verina, agents produce 156 verified improvements among the 189 targets. Using the fastest verified implementation for each target, the geometric mean speedup is $1.851\times$ and the median is $1.117\times$. On 17 migrated repositories, agents produce 92 rewrites proved equivalent to the Lean ports they replace, all faster on function-level microbenchmarks. The geometric mean speedup is $1.864\times$, with a median of $1.402\times$. Fifteen of these optimized functions outperform functions in the original source-language programs. On 51 application scenarios, execution improves in 47 cases with a geometric mean speedup of $1.120\times$ over the migrated Lean ports. The repository rewrites remove repeated normalization and temporary lists, add fast paths for common input cases, and change traversal order while preserving the reference behavior. Together, these findings outline concrete steps toward verified optimization of real-world software, which we discuss in Section 6.

2 BACKGROUND AND RELATED WORK

Existing work on program optimization and verification shares several limitations that prevent agents from optimizing real-world software with guaranteed correctness.

First, checking optimization correctness with tests does not guarantee behavior preservation. PIE [65] generates program edits for speed, and LLM Compiler [11] selects compiler optimizations for code size. At the algorithm level, AlphaDev [46] and AlphaEvolve [51] search for improved algorithms. For GPU kernels, generation and optimization are studied in KernelBench [53], Sakana's CUDA Engineer [35], CUDA-L1 [37], Astra [76], and AsmEvo [39]. At system scale, Jitskit [40] synthesizes systems from specifications. These systems check correctness with tests or differential execution. When this assessment determines the reward for optimization, an agent can improve its score by exploiting the checks or omitting work needed outside the tested cases. Documented failures include exploiting harness bugs or weaknesses in timing methodology [35, 37], specializing to benchmark input distributions [40, 89], and passing standard tests while violating kernel contracts [60, 63]. In agentic verified optimization, an equivalence proof rules this out because it covers every input.

Second, optimizers built on predefined transformations cannot go beyond those transformations. Verified compilers such as CompCert [36] and CakeML [32] apply passes proved correct in advance, and egg [77] explores the consequences of supplied rewrite rules. Superoptimizers [47] such as Denali [31], STOKE [62], and Souper [61] search instruction sequences within a fixed representation, and translation validation [55, 50], implemented for LLVM in Alive2 [43] and HEC [87], checks individual compiler transformations. At the source level, Fiat [15], the Isabelle Refinement Framework [33], and Cogent [52] derive implementations from refinement steps supplied by a developer or compiler, and LLMLift [4] verifies translations of code into domain-specific languages such as Spark and TACO, so any performance benefit comes from the optimizations those languages already provide. Even when every transformation is verified, such an optimizer finds only what its passes, rules, or target language can express, which places a ceiling on its gains. An agent has no such fixed set, since it can write a new algorithm or data representation directly in Lean and prove it equivalent to the original without moving the program to another language.

Third, agents that optimize under formal checks have limited ability to prove their rewrites. LLM-Vectorizer [70], VecTrans [90], and LLM-VeriOpt [22] check LLVM rewrites with Alive2's bounded validation, which accepts only 38.2% of LLM-Vectorizer's test programs, and T-LLM [23], which pairs the bounded model checker CBMC with a language-model judge, reports a 33.8% false-acceptance rate. AutoGate [75] and Ares [13] check hardware rewrites with commercial equivalence tools, CertiBlock [66] checks six predefined rewrite families for Scratch programs, and CktFormalizer [79] proves equivalence for circuits in a domain-specific language embedded in Lean. Other systems let the agent prove its rewrite against a written specification, such as IDS [1] for distributed systems, Solidus [54] for a verified Yul-to-EVM compiler, and lean-zip [49] for a verified DEFLATE implementation. These specifications are hard to write and must be written anew for each system [19, 86]. Our task uses the existing implementation as the specification, so no specification has to be written before a function can be optimized.

Fourth, verified code-generation benchmarks measure correctness but not performance. Function-level benchmarks in Lean, including miniCodeProps [42], FVAPPS [17], Verina [85], and CLEVER [71], benchmarks for SMT-based verifiers such as DafnyBench [44] and VerusBench [8], VeriCoding [9] across Dafny, Verus, and Lean, and repository-scale benchmarks such as RVBench [91], VeruSAGE-Bench [81], VeriSoftBench [78], and Vero [84] all check whether an implementation satisfies a specification, and none evaluates its speed.

Finally, Lean has few executable real-world applications to optimize. Lean is both a proof assistant and a general-purpose language with a native compiler [14, 73], but its software developments are formal models of languages, such as Aeneas [25] and Strata [69], and individual components, such as Cedar [12], lean-zip [49], and RyuLean4 [59], none of which has the callers and workloads of a deployed application. Language models can translate programs between languages [29], with translations checked by bounded model checking in VERT [80] or by differential fuzzing in FLOURINE [20]. AQUEDUCT migrates applications into native Lean and validates each port with differential testing [48], coverage-guided fuzzing [88, 41], and mutation analysis [28].

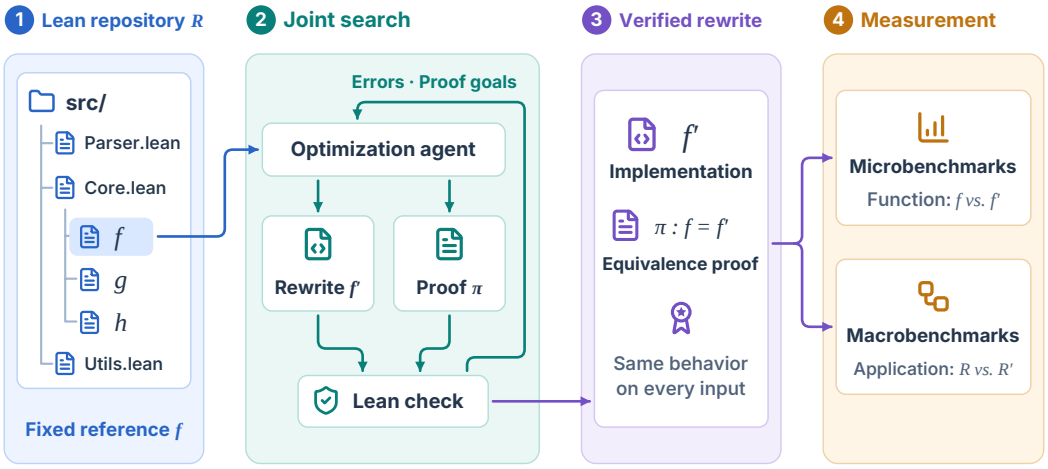


Figure 1: **Agentic verified optimization in a repository.** An agent selects a function f from a Lean repository and jointly searches for a replacement f' and an equivalence proof π. Keeping f fixed, it uses Lean's errors and proof goals to revise both artifacts. The search returns the replacement with a checked proof that it preserves the reference's behavior. Microbenchmarks measure function-level speedup, and macrobenchmarks measure the effect on the application.

3 Agentic Verified Optimization

The task starts with an executable reference and asks an agent to improve its performance while preserving its behavior. For a given function, this requires searching for an implementation together with its equivalence proof. Within a repository, the agent must also choose which functions to optimize (Figure 1).

3.1 Problem Formulation

Program optimization can be expressed as a search for a faster implementation under an equivalence constraint. STOKE [62] uses this formulation to search instruction sequences, approximating equivalence with tests during search and validating the result afterward. We apply the same formulation to source-level programs, with the agent searching for both a replacement and its equivalence proof.

Behavioral equivalence. A reference implementation fixes the behavior that an optimization must preserve. For a pure function, this means returning the same result on every input. For a function whose effects are expressed inside Lean, such as state threaded through a monad or exceptions, the effects are part of the function's value and the same equality applies. In-place updates of uniquely referenced arrays are ordinary pure code in Lean, so representation changes such as replacing lists with arrays or preallocating buffers fall within the task. Opaque effects, such as input and output or foreign calls, whose behavior Lean does not define, fall outside this study, and Section 6 discusses modeling them. We express this obligation in Lean 4 for an executable reference function f in a repository $\mathcal{R}$. The replacement f' has the same signature. Its equivalence certificate is a proof $\pi : f = f'$. By function extensionality, the agent can establish it by proving $\forall x,\ f(x) = f'(x)$. The proof permits replacing calls to f with calls to f' while preserving their results.

Equivalence instead of a written specification. Verified code generation usually states correctness against a formal specification written for the task. Joint implementation-and-proof systems such as IDS [1] and Vero [84] prove that an implementation refines or satisfies such a specification, and an optimizer could in principle be scored against one as well. A specification, however, must first be written and then trusted. Writing one that captures the intended behavior is difficult [19, 86, 45], and manually curated specifications contain errors [84]. A specification that omits part of the intended behavior also lets an agent satisfy it while dropping that behavior, the same reward hacking that

weak tests invite [67]. Optimization has a simpler starting point. The reference implementation is executable, so its behavior can be read and tested directly, and it can be written as the simplest correct algorithm regardless of speed. Equality to it specifies the behavior completely, and the obligation does not change as the agent explores faster algorithms. This is also the setting of regression verification, in which the previous version of a program serves as its specification [24].

Optimization objective. The agent receives the reference and its repository context and searches for a replacement together with a proof. The evaluator fixes a workload $\mathcal{B}$ on which speed is measured. Writing $t_{\mathcal{B}}$ for execution time on that workload, the joint objective is

$$\underset{f',\,\pi}{\text{minimize}} \quad t_{\mathcal{B}}(f') \qquad \text{subject to} \qquad \pi : f = f'. \tag{1}$$

Here the constraint requires Lean's kernel to accept the proof with the reference fixed. The replacement must be executable and build with its callers, and the agent may supply auxiliary definitions and lemmas needed by either artifact. Section 3.3 specifies these checks. Within a finite search budget, a successful result is a feasible pair with $t_{\mathcal{B}}(f') < t_{\mathcal{B}}(f)$.

Function-level optimization. The agent is given a target f and seeks a faster implementation with an accepted equivalence proof.

Repository-level optimization. The agent receives the repository and chooses which functions to optimize within its search budget (Figure 1). Each replacement requires an equivalence proof, and the integrated replacements must build with the application. The objective is to reduce application execution time. We write $\mathcal{R}'$ for the repository after integrating the accepted replacements. Function-level microbenchmarks measure individual rewrites, while application-level macrobenchmarks compare $\mathcal{R}'$ with $\mathcal{R}$ (Section 3.3). These measurements distinguish a locally faster function from an improvement to the application that uses it.

Target selection. Target selection determines how repository-level optimization spends its search budget. A function is worth optimizing when it accounts for a large share of application time and its rewrite appears provable. Source code, call structure, and profiles on development workloads indicate the first, and the function's structure and the lemmas already available indicate the second. Targets also differ in granularity. A local rewrite replaces one function, carries one equality obligation, and can gain at most that function's share of runtime. A rewrite of a module or of a shared data representation changes several functions at once, and the obligation then falls on the module's interface, where each exported function must remain equal to its reference while internal helpers may change freely. The larger obligation gives access to representations and control paths that a rewrite of a single function cannot reach. An agent can therefore spend its budget on many cheap local rewrites, on one structural rewrite with a long proof, or on a sequence in which local rewrites establish lemmas that a later structural rewrite reuses, and Section 6 lists the comparisons that would distinguish these strategies.

Workloads and specialization. The workload $\mathcal{B}$ defines what faster means, and the task leaves open how much the agent knows about it. A study can withhold $\mathcal{B}$, provide a sample of it, or describe the kind of workload the function serves, and each choice measures a different capability. Withholding the workload measures improvements that generalize across inputs. Describing the workload permits specialization to that kind of workload, as Jitskit [40] does when it synthesizes a system for a declared workload. Test-based acceptance cannot make such specialization safe, because a specialized implementation can be correct on the workload and wrong elsewhere [89]. Under an equivalence obligation, specialization becomes a controlled choice. If the agent guards a specialized implementation with a decidable input predicate P that describes that kind of workload and falls back to f otherwise, the guarded replacement satisfies the full equality obligation and is exactly correct on every input. If the agent instead deploys the specialized implementation alone, the obligation relaxes to $\forall x,\ P(x) \to f(x) = f'(x)$, which certifies the replacement on every input of that kind rather than on the tested inputs. Both forms keep the correctness argument machine-checked, so the amount of specialization is a parameter of the study rather than a limit of the task. Section 5 states the choice made in our study, and Section 6 discusses specialization as a direction.

(a) Reference implementation

```
def lisOriginal (nums : List Int) : Nat :=
  let max2 (a : Nat) (b : Nat) : Nat :=
    if a > b then a else b

  let rec helper (lst : List Int)
      (prev : Option Int) : Nat :=
    match lst with
    | [] => 0
    | h :: t =>
        let canTake : Bool :=
          if prev = none then true
          else if prev.get! < h then true
          else false
        let withTake : Nat :=
          if canTake then
            1 + helper t (some h)
          else 0
        let withoutTake := helper t prev
        max2 withTake withoutTake

  helper nums none
```

(b) Agent's rewrite

```
def lisTableQuery :
    List (Int × Nat) → Option Int → Nat
  | [], _ => 0
  | (value, len) :: entries, none =>
      Nat.max len
        (lisTableQuery entries none)
  | (value, len) :: entries, some prev =>
      if prev < value then
        Nat.max len
          (lisTableQuery entries (some prev))
      else
        lisTableQuery entries (some prev)

def lisTable : List Int → List (Int × Nat)
  | [] => []
  | value :: values =>
      let entries := lisTable values
      (value, 1 + lisTableQuery entries
        (some value)) :: entries

def lisOptimized (nums : List Int) : Nat :=
  lisTableQuery (lisTable nums) none
```

(c) Equivalence theorem (proof omitted)

```
theorem lis_equiv (nums : List Int) : lisOptimized nums = lisOriginal nums := by
```

Figure 2: **From recursive search to dynamic programming.** A simplified example from Verina task `verina_advanced_33`, showing (a) the reference implementation, (b) the agent's rewrite, and (c) their equivalence theorem, proved in Appendix E.1. The rewrite runs $4{,}467.871\times$ faster on the 261-input corpus.

3.2 Joint Implementation and Proof Search

Each rewrite changes the proof the agent must write. Hoisting a repeated computation requires showing that its value can be reused, a changed traversal requires an invariant over the elements already processed, and a new algorithm requires relating different intermediate states. The agent therefore searches for the implementation and its proof together (Figure 1). After proposing a rewrite, it can optimize further, attempt the proof, prove a supporting lemma, or change the implementation to make the proof easier, guided by Lean's errors and proof goals. A failed proof does not tell the agent whether the rewrite is wrong or the proof is incomplete, so dividing effort between implementation and proof is part of the task.

Figure 2 shows an example from Verina [85]. The reference for `longestIncreasingSubsequence` makes two recursive calls per element, one taking it and one skipping it, and so explores 2^n branches on an increasing list of length n. The agent's rewrite fills a table from the end of the list with the length of the longest increasing subsequence starting at each position, visits $O(n^2)$ entries, and runs more than $4{,}000\times$ faster. The proof is the hard part, because the reference recurses on the remaining list and the previously chosen element, while the table is indexed by position. The agent bridged the two with a lemma relating each table query to the reference's recursive helper for every suffix and every possible preceding element, which makes the induction work for both the take and the skip branch.

Lemmas can also be reused across rewrites. In Gron, a migrated JSON tool (Section 4.2), the proof of the string-validation rewrite `validIdentifier` reuses character-classification lemmas proved for an earlier rewrite.

3.3 Acceptance and Measurement

A rewrite is accepted when Lean's kernel checks its equality theorem against the unchanged reference. Following Vero [84], the check runs in a fresh copy of the repository, so the agent's declarations cannot alter the reference, add axioms, or substitute compiled code for the definition being proved (Appendix A).

Metrics. A function-level microbenchmark times a target in isolation, and an application-level macrobenchmark, which we call an *application scenario*, runs a public entry point on fixed inputs with all accepted rewrites integrated. Function-level speedup is $s(f') = t_{\mathcal{B}}(f)/t_{\mathcal{B}}(f')$, application-level speedup for a scenario w is $t_w(\mathcal{R})/t_w(\mathcal{R}')$, and we summarize both with geometric means and medians. For migrated applications, we also time the matching operation in the original program on the same inputs, which compares the ports with their native originals. To describe how gains are distributed over a fixed set of targets F, let s_f be the best verified speedup of target f, or 0 if it has no verified candidate. Following the fast_p metric of KernelBench [53], we define

$$\mathrm{vfast}_p = \frac{1}{|F|} \sum_{f \in F} \mathbf{1}[s_f > p], \qquad p \geq 1, \tag{2}$$

where $|F|$ is the number of targets and the indicator $\mathbf{1}[s_f > p]$ equals 1 when $s_f > p$ and 0 otherwise. Thus vfast_p is the fraction of targets whose best verified candidate is more than p times faster than the reference, and vfast_1 is the fraction with any verified improvement.

4 Targets for Verified Optimization

Both settings need executable Lean programs to optimize. For function-level optimization, we use the standalone functions in Verina [85] (Section 4.1). Repository-level optimization additionally needs complete applications with callers and workloads, which Lean lacks (Section 2), so we build Aqueduct, an agentic framework that migrates C, C++, Go, and Rust programs into Lean and validates each port against its original (Section 4.2).

4.1 Functions from Verina

Source and construction. Verina contains 189 standalone Lean 4 tasks whose reference implementations prioritize correctness and mostly use lists. We use every reference implementation as f and ask for a faster f' with a proof $\pi : f = f'$, without using Verina's formal specifications, since the reference implementation is the specification.

Workloads. A deterministic generator constructs up to 256 cases per target and adds the official tests. Timing uses these cases, with resource limits on four expensive references (Appendix B). Because the functions are standalone, they are measured with microbenchmarks only.

4.2 Applications Migrated by Aqueduct

Aqueduct obtains Lean applications by migrating existing programs, which gives them three properties that a freshly written corpus would lack. Their code structure, configuration surface, and workloads come from deployed software rather than from a benchmark author. The original program is an executable oracle, so a port can be validated on any input without a written specification, just as the reference implementation later specifies the optimization. The original also serves as a performance baseline for the optimized Lean code. The difficulty is fidelity. A port that compiles and passes the common cases can still drop rare options, boundary inputs, exact diagnostics, and exit conventions, and tests written by the porting agent share its assumptions. Aqueduct therefore develops the port and its test corpus together, and no agent judges its own work.

Migration task. Each task supplies a pinned original program and a declared input domain for its public interface. A Lean port is compared with the original at its library API or executable entry points, which leaves its internals unconstrained, so agents write idiomatic Lean rather than mirror the source architecture. For each input, the comparison checks exit status, output bytes, filesystem effects, and timeout state, using byte equality by default and a declared tolerance only for output that cannot be stabilized, such as floating-point formatting. The port must be native Lean with no foreign-function interface (FFI) or subprocess call to the original, since agreement is meaningless if the port consults the oracle. Migration succeeds when the port agrees with the original on every test in the accumulated corpus T. We report recovery, the fraction of tests passed by the original that the port also passes, measured once on a disjoint hidden test set T_{hidden} that no agent sees, which in our evaluation is the hidden ProgramBench suite (Section 5.2). The resulting port becomes the reference for optimization.

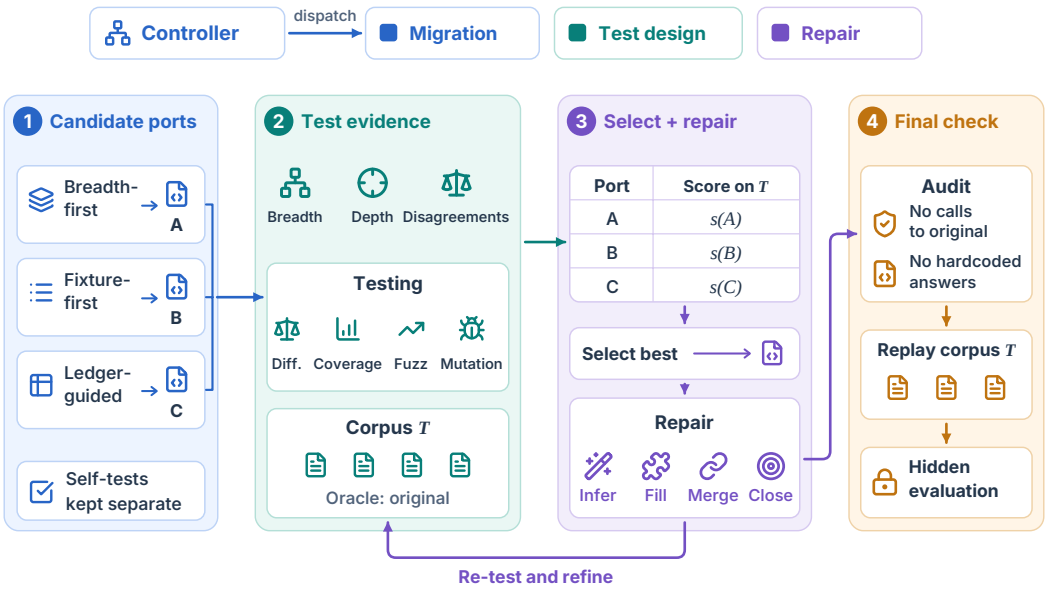


Figure 3: **The AQUEDUCT migration workflow.** Three porting strategies produce candidate implementations. Test-design agents inspect the frozen candidates and build a shared corpus T, using differential execution, coverage, fuzzing, and mutation testing. The controller selects a port by agreement with the original across behavioral families. Repair agents revise the selected port, and rerunning the tests checks each revision as the corpus grows. The controller audits the resulting port for calls to the original and hardcoded answers before a single hidden evaluation.

Migration workflow. Agents do the work that requires judgment, and a deterministic controller makes every decision on the evidence (Figure 3). Three migration agents port each package under different strategies, so that their mistakes differ. One ports the whole package at once, one first groups behavior into families such as a subcommand or an output format and builds test inputs for each family before implementing it, and one keeps an explicit ledger of every family and its status. Once these candidates are frozen, three test-design agents extend the corpus toward uncovered entry points and error paths, structured and boundary inputs, and inputs on which the candidates disagree. They rely on native Lean tools that we build. A differential harness runs both programs on the same input and shrinks each divergence to a minimal counterexample, coverage instrumentation and coverage-guided fuzzing reach untested code, and faults that survive mutation testing mark behavior the corpus executes but does not check. The controller replays the corpus on every candidate and selects the one that agrees most with the original, weighing agreement across behavioral families so that many easy cases cannot hide a missing command or format. Repair agents then fix the selected port against its failing tests, and the controller keeps a fix only if it repairs a failure without breaking a passing test. Part of the evidence is withheld from the repair agents, so every fix is checked on tests its author never saw. Repair alternates with test generation, so each round faces a stricter corpus, until the port agrees with the original on the whole corpus or the budget runs out. The controller finally audits the port for calls to the original and hardcoded answers and measures it once on T_{hidden}.

5 EVALUATION

The study asks whether agents can find faster implementations under an equality-proof requirement and how the improvements affect applications. Section 5.1 measures how often agents succeed on the fixed set of 189 Verina functions and how their gains are distributed. Section 5.2 examines whether local rewrites improve application runtime in the migrated repositories and how well the Lean ports preserve the original programs' behavior. In both studies we use Codex with GPT-5.6 Sol at the `xhigh` reasoning setting, withhold the evaluation workloads from the optimizer, and require full

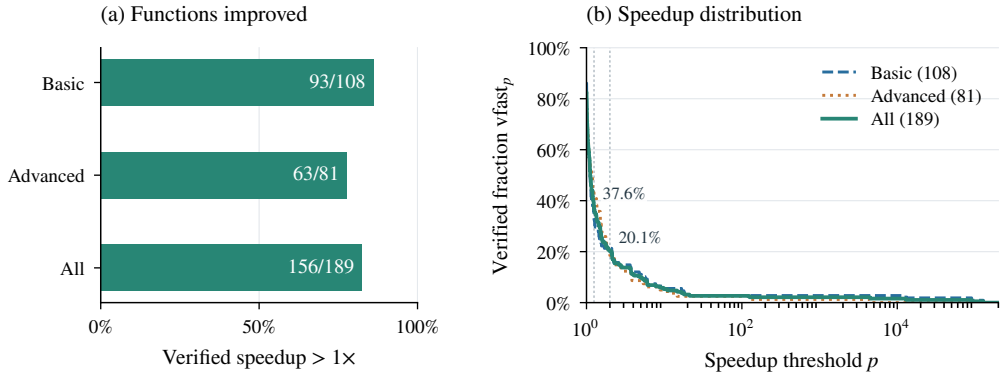


Figure 4: **Function-level optimization on the 189 Verina references.** (a) The fraction of functions with a faster verified implementation, with counts shown inside the bars. (b) vfast_p, the fraction with a verified speedup greater than threshold p (Equation 2), using a logarithmic horizontal axis. Each curve covers every function in its suite.

Table 1: Verified optimization on the 189 Verina functions by suite. Aggregate speedups use the fastest verified implementation for each target, including its reference.

Suite	Targets	Improved	Geom. mean	Median
Basic	108	93	1.959×	1.109×
Advanced	81	63	1.716×	1.163×
All	189	156	1.851×	1.117×

equality to the reference, so the speedups measure improvements that generalize across inputs rather than specialization to the timing inputs.

5.1 Function-Level Optimization on Verina

Setup. The study covers the 189 Verina references, comprising 108 basic and 81 advanced tasks. Each function receives an isolated optimizer invocation containing the reference implementation and the equality theorem to prove. The 54 functions whose initial candidates did not improve performance receive one fresh retry under the same conditions. The timing corpus is frozen before optimization and contains 49,620 generated and official cases. Timing variation can produce apparent improvements even when the program is unchanged. PIE [65] reports a mean apparent speedup of 1.12×, reaching 1.91× in the top 5%, when timing 500 pairs of identical binaries. We therefore use repeated paired trials. The timing harness uses Lean 4.18.0 on an AMD EPYC 9654 processor and is pinned to one logical CPU. After warm-up and iteration calibration, it runs nine paired trials, alternates execution order, and requires matching output digests. Speedup is the baseline median time per corpus pass divided by the candidate median time per corpus pass. Appendix B gives the input restrictions and calibration procedure.

For aggregate performance, we use the fastest verified implementation available for each function, including its reference. Thus, each target contributes $\max(1, s_f)$ to the geometric mean and median, where s_f is the best verified candidate's speedup defined in Section 3.3.

Agents improve 156 of the 189 functions. Agents produce faster implementations for 156 of the 189 targets (82.5%), including 93 basic and 63 advanced functions (Figure 4 and Table 1). The fresh retries contribute 21 of these improvements among the 54 retried functions. The vfast_p curve in Figure 4 shows how these gains are distributed. At higher thresholds, 71 functions exceed 1.25× speedup and 38 exceed 2×, giving $\mathrm{vfast}_{1.25} = 37.6\%$ and $\mathrm{vfast}_2 = 20.1\%$. Over the 189 targets, the resulting implementations achieve a geometric mean speedup of 1.851× and a median of 1.117×.

Table 2: Performance of migrated and optimized applications at both scales. Speedup is baseline runtime divided by evaluated runtime, and values above one favor the evaluated implementation. Faster counts the kernel-checked rewrites or application scenarios whose speedup exceeds one.

	Function level (92 rewrites)			Application level (51 scenarios)		
Comparison	Faster	Geom. mean	Median	Faster	Geom. mean	Median
Optimized Lean vs. migrated Lean	92/92	$1.864\times$	$1.402\times$	47/51	$1.120\times$	$1.077\times$
Optimized Lean vs. original	15/92	$0.235\times$	$0.217\times$	3/51	$0.148\times$	$0.160\times$
Migrated Lean vs. original	12/92	$0.126\times$	$0.120\times$	3/51	$0.132\times$	$0.158\times$

Where agents fall short. The 33 functions without an improvement also have verified candidates, but every such candidate runs slower than its reference, within ten percent of it for 16 functions and at less than half its speed for 2. They are small arithmetic and comparison routines with little work to remove, such as `Abs` and `minOfThree`, and sorting routines such as `insertionSort` and `mergeSort`, whose rewrites were proved equal but not faster. On these functions, the remaining challenge is finding a faster implementation rather than proving it correct. The repository study exposes a complementary challenge in four scenarios where proved local speedups did not improve the application.

Case studies show three ways an agent reaches a faster, provable implementation. The three largest speedups, up to $128{,}292\times$, replace iterated arithmetic sums with closed-form polynomials, and the rewrite of Section 3.2 replaces recursion with dynamic programming. Other cases show how the agent finds a rewrite it can prove. First, the agent reuses verified library algorithms. The reference for `SquareRoot` increments a candidate root until its square exceeds the input, and the rewrite calls Mathlib's logarithmic-time `Nat.sqrt`, running $21.16\times$ faster. The proof shows by induction on the remaining bound that the loop returns `Nat.sqrt N`, maintaining that the candidate is at most the root and the root is at most the candidate plus the bound, and `isPerfectSquare` is rewritten the same way ($17.97\times$). Second, the agent exploits properties of the problem. The reference for `majorityElement` counts every element in a hash map and then searches the table for a count above half the length, while the rewrite returns as soon as one count crosses that threshold, running $3.74\times$ faster. The early return is correct because the input has a strict majority, which is unique, and the proof establishes uniqueness with a lemma that the counts of two distinct elements sum to at most the list length. Third, the agent changes the representation while keeping the control flow. The reference for `nthUglyNumber` builds its sequence by appending to a list and indexing into it, which takes quadratic time, and the rewrite performs the same steps on an array, running $124.90\times$ faster. Because the two loops take identical steps, the proof only relates each array to the list it represents.

5.2 Repository-Level Optimization with Aqueduct

Setup. To evaluate Aqueduct, we apply it to 40 repositories sampled from ProgramBench [83], which supplies pinned C, C++, Go, and Rust programs with hidden end-to-end tests through their command-line interfaces. The repositories range from 258 to 1,524,580 source lines and include archivers, compression libraries, build systems, code analyzers, and geospatial tools. Agents may read the pinned source and execute the original, but cannot access the hidden tests or signals derived from them. Of the 39,857 declared tests, the pinned originals pass 18,642 in the evaluation environment, and we score preservation of their behavior on this original-supported subset. As a baseline, a single migration agent receives the same task and model configuration and produces a port directly. The optimization study runs on 17 of these ports, with 51 application scenarios, three per repository. A separate agent constructs their function-level and application-level workloads from the public entry points and the migration test corpus, and the workloads are fixed before optimization. For each repository, the optimizer inspects the source and call structure to select functions, then proposes replacements and proves them equivalent to the Lean references. We compare migrated Lean, optimized Lean, and the original program on the independently constructed microbenchmarks and macrobenchmarks. Each microbenchmark runs the three implementations on matched function inputs. The macrobenchmarks exercise the application after integrating its accepted rewrites, so their speedups measure the combined effect on each scenario.

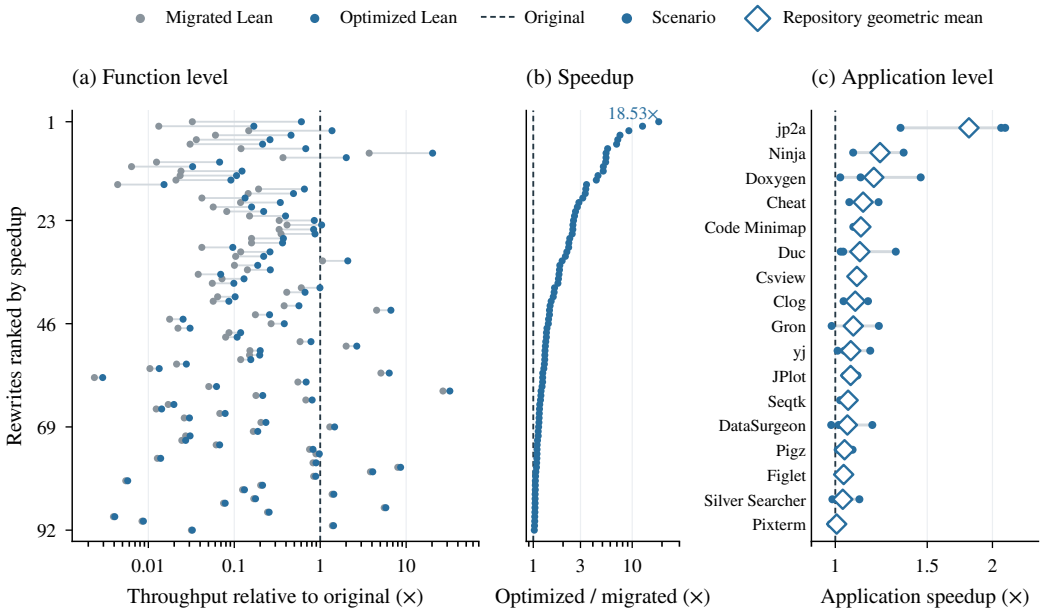


Figure 5: **Repository-level optimization measured at two scales.** (a) Each row is one of the 92 accepted rewrites, ranked by its speedup over the migrated Lean reference, with the migrated and optimized implementations placed by throughput relative to the original source-language operation. The dashed line marks the original. (b) The speedup of each rewrite over its migrated reference, on the same rows. (c) Application speedup of optimized over migrated Lean for the 51 scenarios, grouped by repository, with diamonds marking each repository's three-scenario geometric mean.

Agents produce substantial speedups with proved equivalence. The agents produce 92 proved rewrites, and each is faster than the migrated declaration it replaces (Figure 5a and Table 2). The geometric mean speedup is $1.864\times$, the median is $1.402\times$, and the maximum is $18.53\times$. The rewrites fall into three main patterns. *Algorithmic improvement.* The rewrite replaces the algorithm with one of lower complexity, mainly by removing loops. One migrated function deduplicates a list by filtering and rebuilding it after every insertion, which is quadratic, and the rewrite produces the same list in a single linear pass with a hash set. Another normalizes the same query strings once per item inside a loop, and the rewrite normalizes them once before the loop. *Representation improvement.* The rewrite keeps the algorithm but computes over a cheaper data representation. One rewrite validates a string in place instead of first converting it to a character list, another folds over an array directly instead of converting it to a list, and a third reserves the final buffer size before appending bytes instead of growing the buffer element by element. *Evaluation-order improvement.* The rewrite keeps both the algorithm and the representation but reorders the computation so that inputs the agent expects to be common take the cheap path. One rewrite tests the frequent ASCII cases before falling back to general Unicode predicates, another selects the comparison function once before sorting instead of re-branching on every comparison, and a third returns immediately on a zero exponent instead of entering the multiplication loop. In 15 of the 92 functions, the optimized Lean declaration is faster than the matched original operation, compared with 12 for the migrated declarations. The port overtakes the original when the original performs the same avoidable work, as when the per-item normalization above is hoisted out of the loop and the Lean function goes from $0.37\times$ to $2.01\times$ the speed of the Go original. Where the original's advantage lies in machine-level execution, the rewrites narrow the gap but do not close it. Appendix E examines five representative cases.

The function-level gains survive integration but shrink. The optimized executables are $1.120\times$ faster than the migrated executables, faster in 47 of 51 scenarios, and faster in all 17 repositories by the per-repository geometric mean, yet the improvement is far below the $1.864\times$ function-level gain (Figure 5c). A scenario spends most of its time outside the 92 optimized functions, in I/O, parsing, and runtime paths that the rewrites do not change, so each function speedup is diluted by the rest of

the execution, and the share that remains depends on how heavily the scenario exercises the rewritten functions. Because every integrated rewrite is proved equal to the declaration it replaces, these application-level gains carry the same guarantee as the function-level ones. Closing the remaining end-to-end gap requires optimizing beyond isolated functions, through module-level rewrites of shared representations (Section 3.1) and, for the I/O paths, the extension to opaque effects (Section 6).

AQUEDUCT recovers more source-program behavior than a single agent. To assess the Lean references supplied to repository-level optimization, we compare their behavior with the original programs on the hidden ProgramBench tests. Across the 40 repositories, the AQUEDUCT ports pass 16,345 of the 18,642 original-supported tests, compared with 12,709 for the single-agent baseline. These counts correspond to recovery rates of 87.68% and 68.17%, a difference of 19.50 percentage points. The per-repository comparison shows where the difference comes from. On nine repositories, the single agent's port passes none of the tests, while the AQUEDUCT port recovers between 61.10% and 100%, and these repositories account for 2,685 of the net gain of 3,636 tests. On the twelve other repositories where AQUEDUCT improves, the single agent already reproduces the common core of the program, and the additional 960 tests come from the long tail of rare option combinations, malformed and boundary inputs, exact diagnostics and exit conventions, and environment-dependent behavior that no single porting attempt covers completely. The two methods tie on 14 repositories, all at 83.39% recovery or higher, and the single agent leads on four by at most five tests. AQUEDUCT addresses both the ports that fail outright and the long tail. Independent candidates fail on different behaviors, their disagreements become tests, and the tests drive selection and repair. The additional tests generated during migration also detect more source-level mutants than the ProgramBench suite in a paired comparison (Appendix C). Removing candidate diversity, test-generation feedback, or repeated repair reduces recovery in the component study (Appendix D).

6 RESEARCH DIRECTIONS

The verified improvements in Section 5 range from removing repeated work and adding fast paths to changing algorithms and representations. They leave open how agents should organize this search, which improvements matter in an application, and how the task extends to more software.

Allocating effort between implementation and proof. A faster candidate may require an invariant that is difficult to discover, whereas a smaller change may admit a short proof. Future studies can compare optimizing before proving, proving after each change, and interleaving implementation and proof construction under matched time and token budgets. Tracking verified speedup throughout a run would measure how quickly each strategy finds a useful replacement and whether further search improves it. Matched runs with and without a proof requirement would show how the requirement changes which transformations agents attempt. Comparing agent models and search strategies under matched budgets would likewise show how verified speedup depends on proving capability.

Reusing proofs across optimizations. The Gron example reuses earlier lemmas to justify a change in string representation. This raises the question of when an agent should invest in a general lemma rather than prove only the current rewrite. Comparing fresh proof search with access to lemmas from earlier optimizations would measure the effect on proof cost and verified improvements. Separating reuse within a repository from transfer across repositories would show how much of the resulting proof knowledge applies beyond the original program. Reuse during software maintenance introduces a further question, since callers and dependencies can change after a rewrite is proved. Studying this setting would test whether agents can adapt earlier optimizations and proofs as an application evolves.

Selecting targets for application performance. The difference between function-level and application-level gains makes target selection a central part of repository-level optimization. A direct next experiment is to compare selection from source inspection with selection informed by profiles on development workloads. A second is to compare local function rewrites with module-level rewrites of a shared representation under matched budgets, measuring proof effort against application-level gain. Evaluation on separate application workloads can then test whether those choices improve total runtime. Comparing individual rewrites with their combinations would expose interactions that isolated microbenchmarks do not measure.

Specializing to a kind of workload. Our study withholds the evaluation workload and requires full equality, so every accepted rewrite is correct on all inputs and none is tuned to the timing inputs. Describing the kind of workload a function serves, or providing a sample of it, would let agents specialize data representations and control paths to that workload, as Jitskit [40] does for whole systems. Section 3.1 shows that the task supports this without giving up a machine-checked correctness argument, through a guarded replacement or the relaxed obligation $\forall x,\ P(x) \to f(x) = f'(x)$. Measuring how much additional speedup specialization yields, and how the choice of the predicate P trades proof effort against coverage of the workload, is a direct extension of the present study.

Learning from optimization trajectories. Optimization trajectories connect candidate programs and measured performance with proof states and verifier feedback. Training on these trajectories could teach an agent which changes to attempt, when to continue a proof, and when to revise an implementation. Unsuccessful attempts could help it distinguish transformations that need a different invariant from those that need a different implementation. Evaluation on held-out repositories under fixed budgets would test whether this training improves verified speedup beyond the programs represented in the training data.

Extending equivalence to opaque effects. Our study covers functions whose effects are expressed inside Lean, that is, pure functions and monadic code over Lean-defined monads such as state and exceptions, because equality of the definitions is then the complete correctness relation and Lean's kernel checks it directly. Opaque effects, such as file and network access, whose behavior Lean does not define, remain outside. They fit the same task once they are modeled. Such operations can be abstracted behind an interface of primitive operations, so that a program becomes a pure function from the responses it receives to the requests it issues and the results it returns, and equivalence becomes equality of those functions. Concurrency requires a relation over the interleavings or traces that callers can observe. Each abstraction fixes which observations a rewrite must preserve and which it may change, such as the order of independent operations or the amount of buffering, and choosing it is part of stating the task for a given codebase. Optimizing the I/O, parsing, and runtime paths that dominate application time in Section 5 requires this extension.

Extending the optimization space. The rewrites in Section 5 change algorithms and data representations at the Lean source level and run on Lean's reference-counted runtime [73], which caps how close they can come to the native programs they were migrated from (Table 2). The equality obligation does not depend on the level at which the replacement is written, so a rewrite could target explicit memory layouts, unboxed representations, or vector instructions, with a Lean-checked proof that the compiled artifact implements the original function and the trusted boundary moved to that program's lowering. Much of the Lean compiler and all of its standard library are Lean programs as well, so verified optimizations of core library functions, registered as `@[csimp]` replacements, would carry their proofs to every caller, and optimizations of compiler passes would extend agent-built verified compilation [58] to performance work, so that agents would improve the system on which their own rewrites run.

Certifying migration. For migrated applications, the original program supplies a further correctness relation to establish. A proof connecting that program to its Lean port could compose with the optimization proof to certify the full path from the original to the optimized implementation. Constructing this proof requires models of the source semantics, memory, and runtime operations that connect the two languages. A shorter route exists where such models are available. Aeneas [25] translates a subset of Rust into pure Lean functions, so optimizing a Rust function and proving the Lean models of the two versions equal would apply the task to code that runs natively, with the translation as the trusted step.

Proving performance properties. The objective in this paper measures execution time on a fixed workload, and the proof says nothing about speed. Asymptotic bounds can be proved in Lean, and CSLib [3] treats time complexity as a computational effect, so that a computation returns its value together with a cost under a declared cost model and correctness and complexity are proved separately. A proved improvement in a bound can still run slower in practice when its constants are large. An explicit cost model [64] would let a rewrite carry both, a proof that it reduces a resource measure

or improves an asymptotic bound and its measured speedup, with the agent relating the original and rewritten computations through their outputs and their costs. Comparing proved bounds with measured runtime across rewrites would show which cost models guide practical optimization and where constants dominate.

7 Conclusion

We formulated agentic verified optimization as a search for a faster implementation and a proof of equivalence to an existing program. Fixing the reference gives the agent a precise correctness obligation while leaving it free to remove work, add fast paths, or replace the algorithm. The study finds verified improvements in standalone Lean functions and in applications migrated by AQUEDUCT. Agents are capable of producing verified optimizations, including rewrites that eliminate intermediate structures or reorder computation. Application-level gains are smaller than function-level gains, and four scenarios do not improve, so a locally faster function does not guarantee a faster application. Target selection should therefore account for each function's share of application runtime. Together, the formulation and findings support studying how agents coordinate implementation and proof search to improve application performance with machine-checked correctness guarantee.

References

[1] Shubham Agarwal, Alexander Krentsel, Shu Liu, Mert Cemri, Audrey Cheng, Rui Meng, Tomas Pfister, Chun-Liang Li, Sylvia Ratnasamy, Aditya Parameswaran, Matei Zaharia, Ion Stoica, and Mohsen Lesani. Inductive Deductive Synthesis: Enabling AI to Generate Formally Verified Systems. *arXiv preprint arXiv:2605.23109*, 2026.

[2] Anthropic. Formalizing Fermat's Last Theorem. `https://www.anthropic.com/research/formalizing-fermats-last-theorem`, 2026.

[3] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The Lean Computer Science Library. *arXiv preprint arXiv:2602.04846*, 2026.

[4] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. Verified Code Transpilation with LLMs. In *Advances in Neural Information Processing Systems*, volume 37, pages 41394–41424. Curran Associates, Inc., 2024.

[5] Rudy Bunel, Alban Desmaison, M. Pawan Kumar, Philip H. S. Torr, and Pushmeet Kohli. Learning to Superoptimize Programs. In *International Conference on Learning Representations*, 2017.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[7] Shuze Chen, Kunal Marwaha, Xiaoyang Lu, Henry Yuen, and Tianyi Peng. Prove2Me: An Open Collaborative Platform for Scaling Math Formalization. *arXiv preprint arXiv:2608.28433*, 2026.

[8] Tianyu Chen, Shuai Lu, Shan Lu, Yeyun Gong, Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Hao Yu, Nan Duan, Peng Cheng, Fan Yang, Shuvendu Lahiri, Tao Xie, and Lidong Zhou. Automated Proof Generation for Rust Code via Self-Evolution. In *International Conference on Learning Representations*, volume 2025, pages 71891–71918, 2025.

[9] Ionel Emilian Chiosa, Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, Miranda Zhao, Max Tan, and Max Tegmark. A Benchmark for Vericoding: Formally Verified Program Synthesis. *Transactions on Machine Learning Research*, 2026.

[10] Berkeley Churchill, Oded Padon, Rahul Sharma, and Alex Aiken. Semantic program alignment for equivalence checking. In *Programming Language Design and Implementation (PLDI)*, 2019.

[11] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. LLM Compiler: Foundation Language Models for Compiler Optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction*, pages 141–153. ACM, 2025.

[12] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):670–697, 2024.

[13] Stef Cuyckens, Mihaela Jivanescu, Jun Yin, Chao Fang, and Marian Verhelst. ARES: Adaptive Reasoning-Effort Steering for PPA- and Cost-Aware RTL Optimization with LLM Agents. *arXiv preprint arXiv:2607.27879*, 2026.

[14] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*, pages 625–635. Springer International Publishing, 2021.

[15] Benjamin Delaware, Clément Pit-Claudel, Jason Gross, and Adam Chlipala. Fiat: Deductive Synthesis of Abstract Data Types in a Proof Assistant. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 689–700. ACM, 2015.

[16] Edsger W. Dijkstra. The Humble Programmer. *Communications of the ACM*, 15(10):859–866, 1972.

[17] Quinn Dougherty and Ronak Mehta. Proving the Coding Interview: A Benchmark for Formally Verified Code Generation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, pages 72–79. IEEE, 2025.

[18] Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. Mercury: A Code Efficiency Benchmark for Code Large Language Models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[19] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering*, 2024.

[20] Hasan Ferit Eniser, Hanliang Zhang, Cristina David, Meng Wang, Maria Christakis, Brandon Paulsen, Joey Dodds, and Daniel Kroening. Towards Translating Real-World Code with LLMs: A Study of Translating to Rust. *arXiv preprint arXiv:2405.11514*, 2025.

[21] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. Simple High-Level Code for Cryptographic Arithmetic – With Proofs, Without Compromises. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1202–1219. IEEE, 2019.

[22] Xiangxin Fang, Jiaqin Kang, Rodrigo Rocha, Sam Ainsworth, and Lev Mukhanov. LLM-VeriOpt: Verification-Guided Reinforcement Learning for LLM-Based Compiler Optimization. In *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 740–755. IEEE, 2026.

[23] Zahra Fazel, Sunanda Gamage, Shayan Shirahmad Gale Bagi, Amir H. Ashouri, Tomasz S. Czajkowski, Bryan Chan, Reza Azimi, and Yaoqing Gao. T-LLM Compiler: Trusted LLM-based Code Optimization and Verification Framework. *arXiv preprint arXiv:2608.14953*, 2026.

[24] Benny Godlin and Ofer Strichman. Regression Verification: Proving the Equivalence of Similar Programs. *Software Testing, Verification and Reliability*, 23(3):241–258, 2013.

[25] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, 6(ICFP):711–741, 2022.

[26] Dong Huang, Yuhao Qing, Weiyi Shang, Heming Cui, and Jie M. Zhang. EffiBench: Benchmarking the Efficiency of Automatically Generated Code. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[27] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sönne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026.

[28] Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[29] Chao Jiang, Dugang Liu, Cheng Wen, Zhiwu Xu, Hua Zheng, Muhammad Sadiq, Jawwad Ahmed Shamsi, Shengchao Qin, and Zhong Ming. Large Language Models for Multilingual Code Intelligence: A Survey. *arXiv preprint arXiv:2604.25960*, 2026.

[30] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[31] Rajeev Joshi, Greg Nelson, and Keith Randall. Denali: a goal-directed superoptimizer. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, pages 304–314. ACM, 2002.

[32] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: a verified implementation of ML. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 179–191. ACM, 2014.

[33] Peter Lammich and Thomas Tuerk. Applying Data Refinement for Monadic Programs to Hopcroft's Algorithm. In *Interactive Theorem Proving*, pages 166–182. Springer Berlin Heidelberg, 2012.

[34] Guillaume Lample, Timothee Lacroix, Marie-Anne Lachaux, Aurelien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. HyperTree proof search for neural theorem proving. In *Neural Information Processing Systems (NeurIPS)*, 2022.

[35] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization. *arXiv preprint arXiv:2509.14279*, 2025.

[36] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[37] Xiaoya Li, Xiaofei Sun, Albert Wang, Jiwei Li, and Chris Shum. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. In *International Conference on Learning Representations*, volume 2026, pages 123922–123957, 2026.

[38] Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, et al. Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction. In *International Conference on Learning Representations (ICLR)*, 2026.

[39] Ji Liu, Puyuan Yang, Rongzhang Zheng, Fan Wang, Jinglin Wang, Muhammad A. Awad, Mortis Huang, Andy Chang, Zekai Li, Zeping Li, Zihao An, Yue Liu, Yuchen Yang, Jianghui Wang, Chushi Chen, Ziqiong Liu, Fuwei Yang, Dong Li, Wen Heng Chung, Shengcai Liu, and Emad Barsoum. AsmEvo: Agentic Assembly-Level Optimization of AMD GPU Kernels with Functional Equivalence Verification. *arXiv preprint arXiv:2608.20711*, 2026.

[40] Shu Liu, Alexander Krentsel, Shubham Agarwal, Mert Cemri, Ziming Mao, Soujanya Ponnapalli, Alexandros G. Dimakis, Sylvia Ratnasamy, Matei Zaharia, Aditya Parameswaran, and Ion Stoica. The Time is Here for Just-in-Time Systems: Challenges and Opportunities. *arXiv preprint arXiv:2605.24096*, 2026.

[41] LLVM Project Contributors. libFuzzer: A Library for Coverage-Guided Fuzz Testing. `https://llvm.org/docs/LibFuzzer.html`, 2026.

[42] Evan Lohn and Sean Welleck. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.

[43] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79. ACM, 2021.

[44] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. DafnyBench: A Benchmark for Formal Software Verification. *Transactions on Machine Learning Research*, 2025.

[45] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 16–28. IEEE, 2025.

[46] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, Taylan Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, 2023.

[47] Henry Massalin. Superoptimizer: a look at the smallest program. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 122–126. ACM, 1987.

[48] William M. McKeeman. Differential Testing for Software. *Digital Technical Journal*, 10(1):100–107, 1998.

[49] Kim Morrison. lean-zip: A Formally Verified DEFLATE Implementation in Lean 4. `https://github.com/kim-em/lean-zip`, 2026.

[50] George C. Necula. Translation validation for an optimizing compiler. In *Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation*, pages 83–94. ACM, 2000.

[51] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[52] Liam O'Connor, Zilin Chen, Christine Rizkallah, Vincent Jackson, Sidney Amani, Gerwin Klein, Toby Murray, Thomas Sewell, and Gabriele Keller. Cogent: uniqueness types and certifying compilation. *Journal of Functional Programming*, 31:e25, 2021.

[53] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47356–47415. PMLR, 2025.

[54] Paradigm. Solidus: A Formally Verified Yul-to-EVM Compiler and Optimization Challenge. `https://github.com/paradigmxyz/solidus`, 2026.

[55] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 151–166. Springer Berlin Heidelberg, 1998.

[56] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

[57] ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

[58] Martin C. Rinard. Testing, Credible Compilation, and Verification in the Axon Verified Compiler in Lean and Claude Code. In *Proceedings of the 2026 ACM SIGPLAN International Workshop on Principles of Agentic Engineering*, pages 12–24. ACM, 2026.

[59] RyuLean4 Contributors. RyuLean4: Formal Verification of the Ryu Float-to-String Algorithm in Lean 4. `https://github.com/lexicone42/ryu-lean4`, 2026.

[60] Dipankar Sarkar. The Correctness Illusion in LLM-Generated GPU Kernels. *arXiv preprint arXiv:2606.20128*, 2026.

[61] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Gratian Lup, Jubi Taneja, and John Regehr. Souper: A Synthesizing Superoptimizer. *arXiv preprint arXiv:1711.04422*, 2018.

[62] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the eighteenth international conference on Architectural support for programming languages and operating systems*, pages 305–316. ACM, 2013.

[63] Rishi Shah and Rishav Shrestha. A Contract-Grade Verifier for LLM-Generated GPU Kernels, and a Native Blackwell Backward for the Gated-Linear-Recurrence Family. *arXiv preprint arXiv:2608.12700*, 2026.

[64] Nikil V. Shyamsunder. Formal Performance and Compile Time Guarantees for Compiler Optimization Heuristics. *arXiv preprint arXiv:2608.20137*, 2026. Accepted to the FMCAD 2026 Student Forum.

[65] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. Learning Performance-Improving Code Edits. In *International Conference on Learning Representations*, volume 2024, pages 47277–47301, 2024.

[66] Yuan Si and Jialu Zhang. Certificate-Carrying Transformation of Event-Driven Block Programs. *arXiv preprint arXiv:2607.00563*, 2026.

[67] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and Characterizing Reward Gaming. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[68] Nat Sothanaphan. Resolution of Erdős Problem #728: A Writeup of Aristotle's Lean Proof. *arXiv preprint arXiv:2601.07421*, 2026.

[69] Strata Contributors. Strata: A Unified Platform for Formalizing Language Syntax and Semantics. `https://github.com/strata-org/Strata`, 2026.

[70] Jubi Taneja, Avery Laird, Cong Yan, Madan Musuvathi, and Shuvendu K. Lahiri. LLM-Vectorizer: LLM-Based Verified Loop Vectorizer. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pages 137–149. ACM, 2025.

[71] Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetzsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. CLEVER: A Curated Benchmark for Formally Verified Code Generation. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc., 2025.

[72] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, Anja Surina, Moritz Firsching, Gergely Bérczi, Francisco J. R. Ruiz, Arun Suggala, Adam Zsolt Wagner, Eric Wieser, Lei Yu, Aja Huang, Miklós Z. Horváth, Andrew Ferraiuolo, Henryk Michalewski, Edward Lockhart, Codrut Grosu, Thomas Hubert, Matej Balog, Pushmeet Kohli, and Swarat Chaudhuri. Advancing Mathematics Research with AI-Driven Formal Proof Search. *arXiv preprint arXiv:2605.22763*, 2026.

[73] Sebastian Ullrich and Leonardo de Moura. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *Proceedings of the 31st Symposium on Implementation and Application of Functional Languages (IFL 2019)*, pages 1–12. ACM, 2019.

[74] Siddhant Waghjale, Vishruth Veerendranath, Zora Zhiruo Wang, and Daniel Fried. ECCO: Can We Improve Model-Generated Code Efficiency Without Sacrificing Functional Correctness? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024.

[75] Yiting Wang, Chenhui Deng, Chia-Tung Ho, Yanqing Zhang, Zhuo Feng, Cunxi Yu, Ang Li, Gang Qu, and Brucek Khailany. AUTOGATE: Automated Clock Gating via Toggling-Aware LLM-based RTL Rewriting. *arXiv preprint arXiv:2606.17461*, 2026.

[76] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A Multi-Agent System for GPU Kernel Performance Optimization. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*, 2025.

[77] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021.

[78] Yutong Xin, Jocelyn Qiaochu Chen, Greg Durrett, and Işil Dillig. VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean. In *Conference on Language Modeling (COLM)*, 2026.

[79] Jing Xiong, Qi Han, Chenchen Ding, He Xiao, Zunhai Su, Chaofan Tao, and Ngai Wong. CktFormalizer: Autoformalization of Natural Language into Circuit Representations. *arXiv preprint arXiv:2605.07782*, 2026.

[80] Aidan Z.H. Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. VERT: Polyglot Verified Equivalent Rust Transpilation with Large Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1453–1463. IEEE, 2025.

[81] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. VeruSAGE: A Study of Agent-Based Verification for Rust Systems. *arXiv preprint arXiv:2512.18436*, 2026.

[82] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[83] John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can Language Models Rebuild Programs From Scratch? *arXiv preprint arXiv:2605.03546*, 2026.

[84] Zhe Ye, Hantao Lou, Yuechun Sun, Peiyang Song, Zhengxu Yan, Timothe Kasriel, Qingyang Zhang, Kaiyu Yang, Soonho Kong, Jingxuan He, and Dawn Song. Vero: Can AI Agents Build Formally Verified Software Repositories? *arXiv preprint arXiv:2608.13522*, 2026.

[85] Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking Verifiable Code Generation. In *International Conference on Learning Representations*, volume 2026, pages 38933–38972, 2026.

[86] Zhe Ye, Aidan Z. H. Yang, Huangyuan Su, Zhenyu Liao, Samuel Tenka, Zhizhen Qin, Udaya Ghai, Dawn Song, and Soonho Kong. Intent-aligned Formal Specification Synthesis via Traceable Refinement. *arXiv preprint arXiv:2604.10392*, 2026.

[87] Jiaqi Yin, Zhan Song, Nicolas Bohm Agostini, Antonino Tumeo, and Cunxi Yu. HEC: Equivalence Verification Checking for Code Transformation via Equality Saturation. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 1181–1196. USENIX Association, 2025.

[88] Michał Zalewski. American Fuzzy Lop. `https://github.com/google/AFL`, 2014.

[89] Yunxiang Zhang, Ping Yu, Jianyu Wang, Max (Xiangjun) Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-Generated Kernels Actually Beat PyTorch? *arXiv preprint arXiv:2607.16241*, 2026.

[90] Zhongchun Zheng, Kan Wu, Long Cheng, Lu Li, Rodrigo C. O. Rocha, Tianyi Liu, Wei Wei, Jianjiang Zeng, Xianwei Zhang, and Yaoqing Gao. VecTrans: Enhancing Compiler Auto-Vectorization through LLM-Assisted Code Transformations. *arXiv preprint arXiv:2503.19449*, 2025.

[91] Si Cheng Zhong and Xujie Si. Towards repository-level program verification with large language models. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages*, pages 27–39, 2025.

[92] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. In *International Conference on Learning Representations (ICLR)*, 2026.

A CORRECTNESS CHECKS

The protocol applies to functions whose effects are expressed inside Lean, including pure functions and computations in monads such as `Id`, `StateM`, and `Except`, and excludes functions whose behavior depends on opaque effects such as `IO` or foreign calls. For these functions, the equality theorem covers every input without a bound on input size or recursion depth. Following Vero [84], the acceptance checker extracts the submitted declarations and inserts them into a fresh copy of the reference environment. It then applies four checks to protect the reference, validate the proof, and ensure that the executable replacement is the definition being proved.

1. **Build.** In the fresh environment, every existing declaration of $\mathcal{R}$, including the reference f and its callers, remains byte-identical, and the submission adds only the replacement f', its auxiliary definitions and lemmas, and the proof π. In a separate integration check, the acceptance checker redirects uses of the target to the proved replacement and checks that the resulting repository $\mathcal{R}'$ builds with all submitted declarations.
2. **Axioms.** The axioms used by π and every auxiliary declaration, including through their dependencies, must be a subset of {`propext`, `Quot.sound`, `Classical.choice`}. In particular, `sorryAx` is forbidden.
3. **Executability.** The rewrite and its agent-supplied computational dependencies must be computable, without `partial` or `unsafe` definitions or `@[implemented_by]` and `@[extern]` substitutions. They use the fixed library and runtime supplied with the task.
4. **Kernel-only checking.** The proof must not use `native_decide`, which appeals to compiled execution to discharge a proof step.

Condition 3 prevents executable substitutions for the definition being proved, while Condition 4 requires proof steps to be checked by the kernel. The protocol also rejects agent-introduced typeclass instances that would change the interpretation of equality for the types involved. The checked proof and its dependencies can be replayed in Lean. Accepted implementations are compiled and timed with the task's fixed Lean compiler, standard library, and runtime.

B ADDITIONAL TIMING DETAILS

Before filtering, each frozen Verina corpus contains between 258 and 270 cases, including the official tests. The timing harness removes inputs that violate the reference's precondition and limits the workload for four expensive references. For `verina_advanced_27`, the combined input-string length is at most 20. For `verina_advanced_31` and `verina_advanced_43`, list length is at most 12. For `verina_advanced_58`, the natural-number argument is at most 5,000. Both implementations run on the same eligible inputs, with between 5 and 269 cases timed per function. The harness calibrates the number of repetitions separately for the two implementations, then normalizes elapsed time to one corpus pass before computing their ratio.

C MIGRATION PIPELINE AND RESULTS

Pipeline. The migration workflow produces the Lean references used by the optimizer. Its three candidate strategies emphasize broad interface coverage, executable fixtures, and an explicit ledger of behavioral obligations. Separate test-design agents inspect frozen candidates and generate inputs for uncovered behavior, boundary cases, and disagreements between ports. The controller compares each port with the original on those inputs, selects a candidate, and coordinates repair. Coverage and mutation feedback guide further test generation. Migration runs in the benchmark's white-box setting. Agents may read the pinned source and execute the original program, but the hidden tests and any signal derived from them stay inaccessible throughout, so the scores measure generalization to behavior that no agent observed. We compare AQUEDUCT against a baseline of one migration agent given the source repository and the migration task, which represents what one-shot agentic migration achieves without the pipeline. Both run under the same setting with the same model access and differ only in whether the pipeline is applied.

Interface comparison. Migration checks the declared interface on $T \cup T_{\text{hidden}}$, which combines the generated corpus and the hidden tests. Unstable behavior, including randomness, host-dependent floating-point formatting, and language-specific error text, is excluded from that interface or handled by its declared comparison rule. Applying the same comparison rule throughout candidate selection, repair, and hidden evaluation keeps the required behavior consistent across the migration process.

Recovery. ProgramBench declares 39,857 tests across the 40 sampled repositories, of which the original programs pass 18,642 in the evaluation environment. Recovery is the fraction of these original-supported tests that the port passes. Across these repositories, the AQUEDUCT ports pass 16,345 tests and the single-agent baseline passes 12,709, or 87.68% and 68.17% recovery. The difference is 3,636 tests, or 19.50 percentage points.

Generated-test quality. The pipeline rests on the premise that generated evidence must discriminate, telling faithful ports apart from unfaithful ones and exposing the faults a weaker suite would miss. We evaluate this premise directly with mutation analysis. After the test suites are frozen, we inject deterministic source-level faults into the original programs, rebuild each mutated checkout, and count only mutants that compile. A mutant is killed only when a test that passes on the unmutated program fails on the mutant. Because the faults are planted after freezing, the audit is independent of the migration search that produced the tests. The test-design agents generated 33,671 behavioral tests across the 40 migration targets.

Generated tests versus the benchmark's own tests. The central question is whether the generated suite observes behavioral differences that the external benchmark does not, so we score the same compiled mutants under both the generated tests and the hidden ProgramBench tests. On 36 compiled mutants across 9 repositories, the generated tests kill 28 (77.8%) against 17 (47.2%) for ProgramBench (Table 3). The asymmetry is sharpest among mutants that exactly one suite catches. The generated tests account for 12 of these 13, ProgramBench for 1 (exact paired binomial $p = 0.0034$). The generated suite discriminates behavioral differences that the benchmark's own tests do not observe, which is why the pipeline generates its evidence rather than borrowing it.

Table 3: Mutants killed by the generated tests and by the hidden ProgramBench tests on the same 36 compiled mutants across 9 repositories.

Outcome	Mutants
Killed by both suites	16
Killed by generated tests only	12
Killed by ProgramBench tests only	1
Killed by neither	7
Total scored	36

Auditing the suite against itself. We also measure the generated suite's absolute kill rate on the full audit population (Table 4). The 95% interval is a repository-cluster bootstrap. These are end-to-end behavioral tests at the program boundary, where a source-level mutant registers only if its effect propagates to observable output, so interface-level kill rates run below what unit-level mutation testing reports. Within the pipeline, each surviving mutant marks a behavior that the corpus executes but does not yet constrain, and it becomes a target for the next round of test generation. The behaviors the suite does not yet check are therefore enumerated, as in the 7zip case study, which itemizes the port's remaining obligations.

Table 4: Mutation audit of the generated suite on the full audit population.

Repositories	Compiled mutants	Killed	Score	Median per repository	95% interval
25	190	101	53.2%	50.0%	39.1% to 67.0%

D MIGRATION ABLATIONS

Figure 6 compares the full pipeline with four component ablations and two baselines. *One repair round.* The pipeline runs candidate generation, test design, and selection in full, but performs a single round of repair instead of alternating repair with evidence generation until convergence. This measures the full repair loop. *No coverage guidance in the testing loop.* The testing loop loses coverage instrumentation, so test generation no longer learns which code paths remain unexercised. This measures the coverage signal. *No mutation feedback in the testing loop.* The testing loop loses mutation testing, so faults the corpus fails to catch no longer become targets for the next round of test generation. This measures the mutation signal. *No candidate portfolio (best single strategy).* A single migration agent ports each package under one fixed strategy instead of three. We run each of the three strategies this way and the figure reports the best one. This measures candidate diversity and selection together. *Baseline (single agent, no pipeline).* A plain agent ports each package. *Fixed phased pipeline.* A variant with the same migration and testing tools exposes them as a fixed sequence of phases. This measures the value of letting agents revisit earlier steps rather than follow a fixed order.

Each component contributes to recovery. The full pipeline recovers 87.68% of the original behavior against 68.17% for a single agent with the same source, toolchain, and time budget, and every component ablation falls between the two, while the fixed phased pipeline falls below the single agent. The following paragraphs attribute this gap to individual components.

Iterative repair outperforms a fixed sequence of phases. The phased pipeline recovers less than even the single-agent baseline, and it contains the same tools as the full pipeline, so the tools are not the bottleneck. Its phases run in a fixed order and never revisit earlier decisions, so when a later phase uncovers behavior the port misses, the agent cannot return to update the behavioral surface it committed to earlier. The agents fill gaps in their ports when they can revisit earlier decisions, but not when a fixed sequence prevents it. This result suggests giving agents freedom over how they work and concentrating rigor in the acceptance criteria and the checks that judge the result.

Candidate diversity provides the largest gain. Committing to the best single migration strategy caps recovery at 72.96%. The selector picks the first strategy for 18 repositories, the second for 20, and the third for 2, so no strategy dominates and the gain comes from having a portfolio and the evidence to choose among it. This also reflects a structural property of the search. Repair is incremental and rarely changes a port's overall structure, so the structural diversity that matters must already exist among the initial candidates.

Both testing signals improve the corpus in complementary ways. Removing coverage guidance or mutation feedback costs 9.80 and 10.05 points, more than the 8.46 points lost by restricting repair to a single round, so the quality of the test corpus matters more than any stage that consumes it. The two signals match their designed roles. Coverage guidance steers test generation toward behavior no test has exercised, and mutation feedback hardens the checks on behavior the corpus already reaches. Repair can only fix what a test exposes, so a weaker corpus leaves divergences undetected and the pipeline treats them as agreement.

E ADDITIONAL CASE STUDIES

E.1 LONGEST INCREASING SUBSEQUENCE

The following theorem and proof establish equivalence for the rewrite in Figure 2. The input `nums` is an arbitrary list of integers, so the theorem covers every such list. The proof quantifies over `prev`, the optional preceding element used by the recursive helper, and shows that querying the computed table returns the same length as that helper.

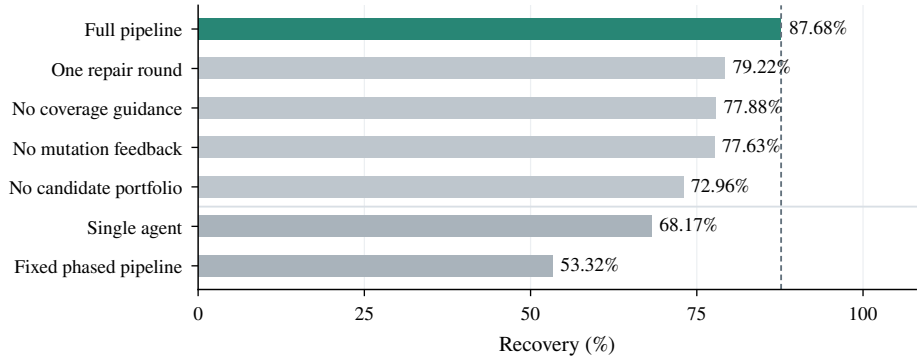


Figure 6: **Contribution of migration components.** Recovery under component ablations and two pipeline baselines. Removing candidate diversity produces the largest decrease among the four component ablations.

```
theorem longestIncreasingSubsequence_optimized_eq_baseline (nums :
    List Int) (h_precond : longestIncreasingSubsequence_precond nums) :

    longestIncreasingSubsequence_optimized nums h_precond =
    longestIncreasingSubsequence nums h_precond := by
  unfold longestIncreasingSubsequence_optimized
    longestIncreasingSubsequence
  clear h_precond
  suffices h : ∀ prev, lisTableQuery (lisTable nums) prev =
      longestIncreasingSubsequence.helper (fun a b => if a > b then a
    else b) nums prev by
    exact h none
  have max_eq (a b : Nat) : Nat.max a b = (if a > b then a else b) :=
    by
    rw [Nat.max_eq_max]
    by_cases hab : a > b
    · rw [if_pos hab, Nat.max_eq_left (Nat.le_of_lt hab)]
    · rw [if_neg hab, Nat.max_eq_right (Nat.le_of_not_gt hab)]
  induction nums with
  | nil => intro prev; rfl
  | cons x xs ih =>
      intro prev
      cases prev with
      | none =>
          simp only [lisTable, lisTableQuery,
    longestIncreasingSubsequence.helper]
          rw [ih (some x), ih none]
          exact max_eq _ _
      | some p =>
          simp only [lisTable, lisTableQuery,
    longestIncreasingSubsequence.helper, Option.some.injEq,
    reduceCtorEq, ↓reduceIte,
            Option.get!, Bool.if_true_right]
          by_cases hp : p < x
          · simp only [hp, ↓reduceIte, Bool.true_eq]
            rw [ih (some x), ih (some p)]
            exact max_eq _ _
          · simp only [hp, ↓reduceIte, Bool.false_eq]
            rw [ih (some p)]
            simp
```

Across the 261 inputs in the evaluation corpus, median time per corpus pass falls from 1.424 seconds to 318.653 microseconds, a speedup of 4,467.871×.

E.2 MIGRATION AT SCALE: 7ZIP

7zip is the fourth largest repository in our sample at 252,968 lines of source, and it tests whether the pipeline can expand a port across a large behavioral surface for frequently used real-world software. ProgramBench declares 883 original-supported tests for 7zip, spanning more than a dozen archive formats and their command-line, stream, and filesystem interactions. The final port recovers 554 of them (62.74%), against 28.77% for the baseline, a direct measure of where migration is hardest. The port is a substantial native Lean implementation. It handles ZIP, TAR, 7z, bzip2, Zstandard, raw LZMA and LZMA86, split archives, AR and DEB, CPIO, RPM, ISO-9660 with Joliet, and CAB with None and MSZip compression, and it implements listing, testing, extraction, creation, update and delete paths, include and exclude filters, standard-stream routing, overwrite behavior, and hash manifests. No single porting attempt produces this surface. It accumulates through the search, as test-design agents surface unhandled formats and options, each divergence becomes a repair target, and each repaired behavior freezes into a permanent regression case. At the final checkpoint the port passes 541 of 579 generated behavioral tests, and the search's inventory of behavioral obligations still lists 938 open items when the budget is exhausted, behaviors the corpus has named but not yet established evidence for. These open obligations account for the gap between the generated-corpus pass rate and hidden-set recovery. The port's remaining gaps are thus measured and itemized, and they direct the next round of the search.

E.3 OPTIMIZATION CASE STUDIES

We examine five proved optimizations that illustrate the three rewrite patterns of Section 5.2.

Algorithmic improvement (Cheat `findConsolidated`). The migrated function first builds the complete consolidated sheet list and then searches it for the requested title, so every lookup pays for constructing entries it immediately discards. The optimized declaration searches from the end of the ordered paths and returns the last matching title directly, never materializing the intermediate list. The equality theorem proves that direct reverse search returns exactly the element the build-then-search pipeline would find, folding the consolidation semantics into the lookup. The migrated declaration is already faster than the matched Go operation, and the rewrite, $5.46\times$ faster than migrated Lean, widens that margin to $20.36\times$ by removing the build-then-search work that the Go code also performs.

Algorithmic improvement (Cheat `consolidatedSheets`). The migrated function builds its result by filtering the entire accumulated list before every append, repeatedly traversing and reallocating a growing structure. The optimized declaration walks the input once in reverse, tracks seen titles in a hash set, and constructs the output list once. The equality theorem proves that the reverse traversal preserves the exact last-title-wins semantics of the original fold, a property that is easy to get wrong in a manual rewrite. The rewrite is $1.46\times$ faster than migrated Lean and $6.65\times$ faster than the matched Go operation.

Representation improvement (Gron `validIdentifier`). The migrated function materializes a full character list to check an identifier. The optimized declaration reads the first character directly and checks the remainder through a string slice, allocating nothing. The theorem composes the slice-to-list equivalence with previously proved character-classification replacements, illustrating how proved rewrites accumulate into a reusable equality library. The rewrite is $1.96\times$ faster than migrated Lean and $2.10\times$ faster than Go.

Algorithmic improvement (Cheat `filterSheets`). The migrated function re-trims every requested tag once per sheet. The optimized declaration normalizes the tags once and reuses the prepared list across all sheets, changing repeated normalization into a single pass. The rewrite is $5.45\times$ faster than migrated Lean and $2.01\times$ faster than the matched Go operation, and the output checksums agree across all three implementations.

Evaluation-order improvement (Code Minimap `pow10`). The migrated helper enters its multiplication loop even for a zero exponent. The optimized declaration returns the zero case immediately, producing a $9.29\times$ within-Lean speedup and running $1.37\times$ faster than the Rust original on a microbenchmark whose inputs include the zero exponent. The equality theorem shows that the fast path

changes no result, because the declarations with and without the fast path are equal on every input. Under the equivalence obligation, the agent can therefore add fast paths for inputs it expects to be common without restricting correctness to those inputs.

In all five cases, the rewrite targets a pattern that idiomatic porting tends to produce, namely repeated filtering, intermediate materialization, redundant normalization, or a missed fast path, and the equality proof guarantees that removing the pattern preserves behavior on every input.